



endingo

정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 이정훈입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

 Contact Me

Data science/Langchain

[Langchain] 2. 모델 Fine_Tuning

endingo 2025. 5. 1. 15:16 ⋮

Fine-Tuning

-사전 훈련된 모델을 특정 작업이나 데이터셋에 맞게 미세 조정하는 과정

파인튜닝 수행 절차

- 사전 훈련된 모델 선택
- 데이터 준비
- 모델 수정
- 추가 학습

1. 데이터 준비

특정 작업에 사용할 데이터 준비

사전 훈련된 모델과 호환되는 형태로 전처리

NLP: 텍스트 토큰화

컴퓨터 비전: 이미지를 적절한 크기로 resize



정훈이의 공부일지



```
train, val = train_test_split(data, test_size=0.2, random_state=42)
```

데이터셋 만들기

```
from datasets import Dataset
```

```
# df로 부터 텐서 데이터셋 만들기
```

```
train_ts = Dataset.from_pandas(train)
```

```
val_ts = Dataset.from_pandas(val)
```

2) 토크나이징

토크나이징 결과:

- input_ids: 토큰화 된 입력 시퀀스를 숫자 ID로 변환한 것

- attention_mask: 모델이 패딩 된 부분을 무시하고 실제 유용한 데이터만 집중

```
def preprocess_function(data):
```

```
    return tokenizer(data["text"], truncation=True, padding=True)
```

2. 파인튜닝

사전 훈련된 모델을 특정 작업이나 데이터셋에 맞게 미세조정하는 과정

1) 사전학습 모델 준비

AutoModel: 사전 훈련된 모델의 이름이나 경로 제공 -> 가중치를 자동으로 로드

SequenceClassification: 시퀀스 분류 => 다중 분류

num_labels: output_layer의 노드 수(다중분류 클래스 수)

```
n = 6
```

```
model = AutoModelForSequenceClassification.from_pretrained(model_name,
                                                            num_labels = n).to(device)
```

2) 학습

- TrainingArguments 설정

Hugging face trainer의 학습 세부 설정을 컨트롤

```
training_args = TrainingArguments(
```

```
    output_dir = './results',
```

```
    eval_strategy = "epoch",
```

```
    save_strategy = "epoch",
```

```
    learning_rate = 2e-5,          # 작은 학습률
```



정훈이의 공부일지



```
weight_decay = 0.02,          # weight decay
load_best_model_at_end = True, # earlystopping 사용하기 위해 필요
logging_dir = './logs',
logging_steps = 10,
report_to="tensorboard"
)
```

- Trainer 설정

모델 학습과 평가를 쉽게 할 수 있도록 도와주는 클래스, API

Trainer 설정

```
trainer = Trainer(
    model=model,          # 학습할 모델
    args=training_args,   # TrainingArguments
    train_dataset = train_ts,
    eval_dataset = val_ts,
    tokenizer = tokenizer,
    callbacks = [EarlyStoppingCallback(early_stopping_patience=3)], # 조기 종료
)
```

- 모델학습

```
trainer.train()
```

- 모델평가

```
eval_results = trainer.evaluate()
print(f"Evaluation results: {eval_results}")
```

3. 허깅페이스에 모델등록

1) 구글 드라이브에 모델 저장하기

- 구글드라이브에 fine_tuned 폴더 생성

```
fine_tuned_path = path + 'fine_tuned_model'
model.save_pretrained(fine_tuned_path)
tokenizer.save_pretrained(fine_tuned_path)
```

2) 허깅페이스에 모델 등록

- API 파일을 Hub에 푸시

- 웹사이트를 통해 파일을 Hub로 업로드(모델 저장소 만들고 파일 업로드)



정훈이의 공부일지



- 모델 다운로드: pipeline 함수를 이용해서 모델을 다운로드
- 모델 사용: 입력(문장), 출력(감정분류 결과)

```
from transformers import pipeline
emotion_classifier = pipeline(task = 'text-classification', model = "")

emotion_classifier('I am really happy to see you again!')
```

4. LoRA 기반 파인튜닝

기존에 학습된 내용은 유지하면서 새로운 데이터에만 필요한 변화만 살짝 추가하는 방식
기존 가중치와 다른 차이만 반영

```
# 기본 설정
model_name = ""
num_labels = 6
tokenizer = AutoTokenizer.from_pretrained(model_name)

# 사전 학습된 모델 로드 (전체 파라미터는 그대로)
base_model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=num_labels)
```

```
lora_config = LoraConfig()
```

작업 유형: 시퀀스 분류 (SEQ_CLS)

랭크 (r): 8 → 적은 수의 파라미터로 학습

스케일 계수 (lora_alpha): 32 → 학습 효과 조절

드롭아웃 (lora_dropout): 0.1 → 과적합 방지

적용 레이어: q_lin, v_lin → attention 레이어에만 적용

Bias 학습: 하지 않음 ("none")

즉, attention 레이어의 일부만 가볍게 학습하도록 최적화한 효율적인 파인튜닝 구성

공감



정훈이의 공부일지



정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

댓글 0

endingo

내용을 입력하세요.

등록

공지사항

최근에 올라온 글

- [Langchain] 2. 모델 Fine_Tu...
- [Langchain] 1. 모델 그대로 사...
- [RAG] 2. 벡터 DB구성과 RAG ...
- [RAG] 1. 벡터 DB 구성과 RAG ...

최근에 달린 댓글

- 잘 읽고 갑니다
- 오늘도 화이팅!!!
- 덕분에 좋은 정보 얻어 갑니다 ! ...
- 좋은 글 잘 보고 가요! 감사합니...

Total

268

Today	2
Yesterday	2

링크

TAG

[more](#)

- github
- git

« 2025/05 »						
일	월	화	수	목	금	토
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

글 보관함

- 2025/05 (3)
- 2025/04 (34)
- 2025/03 (7)



정훈이의 공부일지



AI와 엑셀, 데이터 분석을 좋아하는 개발자입니다.

다양한 문제 해결과 창의적인 개발을 추구합니다.

 이메일 보내기